

Control Flow & Functions

Why this matters?

Most real programs are: decisions + repetition + reusable building blocks.

This file focuses on:

- Decisions: `if/elif/else`, `match/case` (and *why* `match/case` can be clearer than long `if/elif` chains)
- Repetition: `for/while`, `break/continue`, `enumerate`, `zip` (tuple-unpacking)
- Reuse: functions (parameters, defaults, docstrings, type hints, multiple returns)

Notes for learners

- `elif` must immediately follow the `if` it belongs to (only comments/blank lines may be in between).
- Truthiness: `0`, `0.0`, `''`, `[]`, `{}` are `False`; non-zero numbers and non-empty containers are `True`. Try: `print(bool(0), bool(2))` to see it.
- Prefer clarity over cleverness. We'll show both clear and compact patterns when helpful.
- Safety: For `while True` loops, we include a small safeguard counter to avoid accidental infinite loops.
- Python 3.10+ is assumed for `str | None` type hints and `match/case`. On 3.9 use `Optional[str]` and `if/elif`.

if/elif/else

Syntax:

```
if (condition1):
    do action1           # executed if 'condition1' == True
elif (condition2):
    do action2
...
else:
    do action{N}         # action{N} with condition1,
```

condition2, ..., condition{N-1} all being False

```
In [1]: test_input = 'Jin'

# `isinstance()` is a function that checks if an input is equal to
# a type of interest
# (ex. `isinstance([1, 2, 3], list)`      -> True)
# (    `isinstance({'A': 1, 'B': 2}, dict)` -> True)
if isinstance(test_input, str):
    # `isinstance(test_string, str)` is True, so below will run.
    # Again, we see the use of F string here.
    print(f"Input is a string: {test_input}")

# You can have as many `elif`'s as you need/want
# If you want the line below to be executed, what would you do?
# Can you change test_input to an integer value instead of 'Jin'?
# (ex. 8)
elif isinstance(test_input, int):
    print(f"Input is an integer: {test_input}")
# If `test_input = [1, 2, 3]`, printed output will be:
# "Input is a list: [1, 2, 3]"
elif isinstance(test_input, list):
    print(f"Input is a list: {test_input}")
# Below is not necessary when you don't want to do
# anything with False condition.
# Try switching `test_input` to a dict and run this 'cell' again.
else:
    print("Eh, input is not a string")
```

Input is a string: Jin

if/else in list comprehension

- Binary choice (only consider True cases):
`[(action) for x in {iterable} if (condition)]`
- Additional treatment specified:
`[(action1) if (condition) else (action2) for x in {iterable}]`

```
In [2]: # Only consider True (i.e. non-zero numbers) cases
integers = range(10)
# Odd numbers or even numbers?
odd_or_even = [x for x in integers if x % 2]
```

% is a modulo operator in Python (ex. `3 % 2 = 1`).

1. {iterable} is `range(10)`. This means `x` will be 0, 1, 2, ... 9 at each iteration
2. (action) is `x`, which means to *append* `x` to a list.
3. (condition) is `x % 2`. At each iteration, if this is `True`, append `x`.

In **boolean indexing**, 0 is `False`, and any non-zero number is `True` (yes, not only 1, but 2, 3, etc.).

- If `x` is even, then `x % 2 == 0` (`False`) -> `x` not appended to the list.
- If `x` is odd, then `x % 2 == 1` (`True`) -> `x` appended to the list.

In [3]: `odd_or_even`

Out[3]: `[1, 3, 5, 7, 9]`

You can include `else` in a (list) comprehension.

In [4]: `# 'else' in a list comprehension`
`odd_or_even2 = ['Odd' if y % 2 else 'Even' for y in integers]`
`odd_or_even2`

Out[4]: `['Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd']`

`elif` is not directly used in a list comprehension. Instead, you 'nest' an `if-else` condition inside an `else` condition.

- Typical `if/elif/else` statement:

```
a_list = list()
for x in {iterable}:
    if (condition1):
        a_list.append(action1)
    elif (condition2):
        a_list.append(action2)
    else:
        a_list.append(action3)
```
- List comprehension's case:

```
a_list = [(action1) if (condition1) else (action2) if
(condition2) else (action3) for x in {iterable}]
```

In [5]: `#@title First ladies again: run to read csv`
`import pandas as pd`
`url = 'https://raw.githubusercontent.com/ohspc89/reprorehab2025' \`
 `'/students/contents/week3/first_ladies.csv'`
`first_ladies = pd.read_csv(url)`
`first_names = first_ladies.First_name.to_list()`
`last_names = first_ladies.Last_name.to_list()`

In [6]: `# Showing the DataFrame we're reading`
`first_ladies.head()`

Out [6]:

	First_name	Last_name	Maiden_name	year_start	year_end
0	Martha	Washington	Dandridge Custis	1789	1797.0
1	Abigail	Adams	Smith	1797	1801.0
2	Martha	Jefferson Randolph	Wayles Skelton	1801	1809.0
3	Dolley	Madison	Payne Todd	1809	1817.0
4	Elizabeth	Monroe	Kortright	1817	1825.0

In [7]: `# First names of the first ladies`
`first_names[0:5]`

Out[7]: ['Martha', 'Abigail', 'Martha', 'Dolley', 'Elizabeth']

In [8]: `# What would `a_list` be like?`
`# 1) Append "MARTHA!" if a value of the iterable:`
`# `first_names` is "Martha"`
`# 2) Append "Dr.Biden" elif a value of the iterable:`
`# `first_names` is "Jill"`
`# 3) Append None if a value of the iterable is`
`# neither "Martha" nor "Jill".`
`a_list = ["MARTHA!" if x == "Martha" else "Dr.Biden"`
`if x == "Jill" else None for x in first_names]`
`# `filter` is a function that can filter elements.`
`# Here, we're filtering out `None` from `a_list`,`
`# just to remove many `None`.`
`list(filter(None, a_list))`

Out[8]: ['MARTHA!', 'MARTHA!', 'Dr.Biden']

match/case

This is a new feature in Python >=3.10 - useful when your are interested in matching patterns.

In [9]: `test_list = list()`
`for name in first_names:`
`match name:`
`case "Martha":`
`test_list.append("MARTHA!")`
`case "Jill":`
`test_list.append("Dr.Biden")`
`# Similar to if-elif-else, you can skip 'else' condition`
`# you can use `_` - this is similar to `else``
`case _:`
`continue # move on`

```
test_list
```

```
Out[9]: ['MARTHA!', 'MARTHA!', 'Dr.Biden']
```

Loops (for/while)

By now who have seen many `for` loops. The idea is simple, and here I'd like to *revisit* (sorry that I've used these methods already earlier without much explanation) methods that can go well with a `for` loop.

- `enumerate()` : adds a counter
- `zip()` : takes multiple {iterables} and aggregates their elements to a tuple.

```
In [10]: # enumerate - if you need a counter
for i, col in enumerate(first_ladies.columns):
    print(f"Column {i+1}: {col}")
```

```
Column 1: First_name
Column 2: Last_name
Column 3: Maiden_name
Column 4: year_start
Column 5: year_end
```

```
In [11]: for j, row in enumerate(first_ladies.values):
# Boolean indexing (0: False, non-zero: True)
# `not` reverses the boolean value,
# so `not 0 (False)` is `True`.
# Therefore, we're only considering values of j that
# satisfy `j % 10 == 0`, except `j != 0` (not equal to).
if (not j % 10) and (j != 0):
# We're still using `first_ladies`,
# so printed row content will be:
# 'first name; last name; maiden name; year_start; year_end'
    print(f"row {j}: {row}")
```

```
row 10: ['Julia' 'Tyler' 'Gardiner' 1844 1845.0]
row 20: ['Lucretia' 'Garfield' 'Rudolph' 1881 1881.0]
row 30: ['Grace' 'Coolidge' 'Goodhue' 1923 1929.0]
row 40: ['Nancy' 'Reagan' 'Davis' 1981 1989.0]
```

```
In [12]: # zip(A, B, ...)
# At each iteration, elements of `first_names` and
# `last_names` will be aggregated into a tuple
# (ex. ('Martha', 'Waghtington'))
for i, full_name in enumerate(zip(first_names, last_names)):
    print(f"First name: {full_name[0]}; last name: {full_name[1]}")
# Just print first 5 full names
if i >= 5:
    break
```

First name: Martha; last name: Washington
 First name: Abigail; last name: Adams
 First name: Martha; last name: Jefferson Randolph
 First name: Dolley; last name: Madison
 First name: Elizabeth; last name: Monroe
 First name: Louisa; last name: Adams

break/continue

`break` will break the loop. `continue` is to move to the next iteration without any action performed.

```
In [13]: # break a loop in the middle
# `first_names` has 48 elements, so `i` can iterate over range(48)
for i, name in enumerate(first_names):
    if name == "Nancy":
        print(f"Nancy found! Closing the loop at the iteration: {i}")
        # iteration terminates when `name == "Nancy"`
        break
    else:
        # If `name == "Nancy"` is False, do nothing and
        # move to the next iteration.
        continue
```

Nancy found! Closing the loop at the iteration: 40

Use `while` when you don't know the number of iterations needed.

`for` loops will end when it reaches the last iteration index. If you need to loop while a condition is `True`, use `while`.

```
In [14]: # Let's suppose you don't know the index of the first name
# "Lucretia" in the list: `first_names`

to_lookfor = 'Lucretia'
i = 0 # A start index is typically set to 0 - the starting index
while first_names[i] != to_lookfor:
    i += 1 # `i += 1` is the shorthand of `i = i + 1`

# Remember, Python starts counting from 0, not 1
print(f"{to_lookfor} is the {i+1}th first name")
```

Lucretia is the 21th first name

```
In [15]: # While loop can be "broken" as well

k = 0
counter = 0
# Don't do this - it can lead to an infite loop...
```

```

while True:
    if first_names[k] == "Martha":
        counter += 1
        # If "Martha" is found twice, break the loop
        if counter == 2:
            break
        k += 1

print(f"The second 'Martha' is found in row: {k}")

```

The second 'Martha' is found in row: 2

You may want to provide a safe guard to avoid infinite loops if your code is not correctly written.

```

max_iter = 1000 # limit the number of iterations
steps = 0

while True:
    ... # whatever action you want to loop over
    steps += 1 # increase the counter
    # if the loop iterates over `max_iter` times,
    # it will raise an error:
    if steps > max_iter:
        raise RuntimeError("Possible infinite loop-check
        your logic.")

```

If you have convoluted / nested logics, use **for** loops. Whenever possible, however, use **comprehensions** or even **vectorized** methods provided by *numpy* or *pandas*.

Your code may look *longer*, but it will be *faster* if your data is BIG.

```

In [16]: # Every 10th rows?
first_ladies.iloc[[x for x in range(10, len(first_ladies), 10)], :]

```

```

Out[16]:

```

	First_name	Last_name	Maiden_name	year_start	year_end
10	Julia	Tyler	Gardiner	1844	1845.0
20	Lucretia	Garfield	Rudolph	1881	1881.0
30	Grace	Coolidge	Goodhue	1923	1929.0
40	Nancy	Reagan	Davis	1981	1989.0

```

In [17]: # Lucretia, 20th first name
# Remember `.str.match()`?
first_ladies[first_ladies["First_name"].str.match("Lucretia")].index

```

Out[17]: Index([20], dtype='int64')

In [18]: *# Let's suppose you have a n-by-3 accelerometer reading.*
import numpy **as** np

`.tolist()` is a numpy ndarray's method -> make a list
N = 100
 sample_acc_x = (np.random.random(N) + 9.8067).tolist()
 sample_acc_y = (np.random.random(N)).tolist()
 sample_acc_z = (np.random.random(N)).tolist()

In [19]: *# Let's think about calculating the root mean square:*
*# sqrt(x**2 + y**2 + z**2)*

to assess performance
from time **import** time

Ok, so we learned how to use `zip` with a `for` loop!
 acc_rms_for = list()
for acc **in** zip(sample_acc_x, sample_acc_y, sample_acc_z):
 acc_rms_for.append(
 float(np.sqrt(acc[0]**2 + acc[1]**2 + acc[2]**2)))

Numpy already has `numpy.linalg.norm()` method.
First prepare N-by-3 ndarray.
 acc_xyz = np.hstack((np.array(sample_acc_x).reshape(-1, 1),
 np.array(sample_acc_y).reshape(-1, 1),
 np.array(sample_acc_z).reshape(-1, 1)))

Single line
 acc_rms_np = np.linalg.norm(acc_xyz, axis=1)

In [20]: *#@title Benchmark*
from time **import** time

def loop_performance(x, iter=100):
 op_times = list()
 for _ **in** range(iter):
 tic = time()
 acc_rms_for = list()
 for acc **in** zip(x[0], x[1], x[2]):
 acc_rms_for.append(
 float(np.sqrt(acc[0]**2 + acc[1]**2 + acc[2]**2)))
 op_times.append(time() - tic)
 return (np.mean(op_times), np.std(op_times))

def numpy_performance(x, iter=100):
 op_times = list()
 for _ **in** range(iter):
 tic = time()
 acc_rms_np = np.linalg.norm(x, axis=1)


```

    op_times.append(time() - tic)
    return (np.mean(op_times), np.std(op_times))

```

```

In [21]: # Mean processing time for N = 100: ~2e-4
loop_performance((sample_acc_x, sample_acc_y, sample_acc_z))

```

```

Out[21]: (np.float64(0.00035898208618164064), np.float64(0.0006499194651070247))

```

```

In [22]: # Mean processing time for N = 100: ~9.7e-6
# About x100 faster than `for` + `zip`
# This gap will only increase as your dataset size increases.
numpy_performance(acc_xyz)

```

```

Out[22]: (np.float64(2.1533966064453123e-05), np.float64(1.652682370593663e-05))

```

Functions

Start when you see typing the same operation again and again.

At some point, you will start by preparing bunch of functions before conducting an actual analysis (modularization).

The structure of a function is this:

```

def some_function(parameter):
    what a function does

```

```

In [23]: # Functions may not have any parameters (=arguments).
def print_hello():
    print("Hello!")

print_hello()

```

Hello!

```

In [24]: # Function parameters can have default values:
# Here, default values of the specified by:
# `(name of a parameter)=(default value)`
# So here, the default of `num1` is `None`.
# The same is true for `num2`.
def add_nums(num1=None, num2=None):
    # `all(iterable)` returns True if everything in iterable is True.
    # So here, if `num1` is None and `num2` is None, it will be True
    if all((num1 is None, num2 is None)):
        # If conditional is True, return `None`.
        # A function 'ends' with a `return` command.
        return None
    # This is why you don't need explicit `else` statement here.

```

```

# If `if` statement above is true, there's already
# an associated `return` statement,
# so the function will no longer proceed.
# If `if` statement is not true,
# following `return` statement will run.
return num1 + num2

# This returns None, because no `num1` or `num2` value is provided.
# Both parameters will be `None`, and the function will return `None`
# -> so you don't see anything returned
add_nums()

# If `num1` is not None or `num2` is not None,
# `num1 + num2` is returned.
add_nums(3, 7)

```

Out[24]: 10

There's one more concept here: *type hints*. Type hints are annotations that specify the expected data types for variables and function parameters, improving code readability and helping to catch errors before runtime.

Type hints are provided using `:`.

```

def some_function(param:str):
    ...
    return ...

```

Now the function `some_function` will expect a parameter `param` to be a string. However, even if you provide a different data type, it will still proceed. Again, *type hints* are mostly for readability.

```

In [25]: def some_function(param:str):
          return f'param is string: {isinstance(param, str)}'

          some_function(3)

```

Out[25]: 'param is string: False'

```

In [26]: # Both a hint and a default value can be provided simultaneously.
          def print_names(name:str="Jin"):
              return f"Your name is: {name}"

          # This will use the default parameter value: "Jin"
          print_names()

```

Out[26]: 'Your name is: Jin'

```

In [27]: # You can also provide other parameter values

```

```
print_names("Rumi")
```

```
Out[27]: 'Your name is: Rumi'
```

```
In [28]: # Type hint choice can be more than one - use OR(|) operator
```

```
def multiple_type_hints(param:int|float=3.0):
    return param
```

```
# This will return the default value: 3.0
print('param default data type: int (=3)')
print(multiple_type_hints())
print('-----')
print('param data type: int (=7)')
print(multiple_type_hints(7))
print('-----')
print('param data type: float (=2.781)')
print(multiple_type_hints(2.781))
print('-----')
```

```
param default data type: int (=3)
```

3.0

```
param data type: int (=7)
```

7

```
param data type: float (=2.781)
```

2.781

```
In [29]: # Hints are useful when users are not sure of what parameters to give
# to a function. You can read that the function: `print_names` expect
# a string input.
```

```
help(print_names)
```

```
Help on function print_names in module __main__:
```

```
print_names(name: str = 'Jin')
```

```
# Both a hint and a default value can be provided simultaneously.
```

Docstring

A docstring is a string literal that occurs as the first statement in a module, **function**, class, or method definition.

Use triple double quotes (""") around a docstring.

- Single line docstring: `"""this is a docstring."""`
- Multiple line docstring:\n`"""\n`

```
This is a multi-line docstring.
```

```

Right?
Yeah right.
"""

```

```

In [30]: # It's also recommended that you provide 'docstring'.
# A docstring is marked by ''' or """

# Example:
def fortune_teller(question:str|None=None):
    """
    This is a docstring prepared following the NumPy style.
    There are other types: Google style, Sphinx style

    fortune_teller is a function that tells you the answer
    to the greatest question of Life, the Universe, and Everything.

    Parameters
    -----
    question: str|None
        User's question; default to None

    Returns
    -----
    42 (ref. The Hitchhiker's Guide to the Galaxy)
    """
    return 42

fortune_teller()

```

Out[30]: 42

```

In [31]: # Again, check out the norm
# This is the NumPy style of docstring.
# Other docstring styles exist: Google style, Sphinx style
help(np.sum)

```

Help on `_ArrayFunctionDispatcher` in module `numpy`:

`sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)`

Sum of array elements over a given axis.

Parameters

`a` : array_like

Elements to sum.

`axis` : None or int or tuple of ints, optional

Axis or axes along which a sum is performed. The default, `axis=None`, will sum all of the elements of the input array. If `axis` is negative it counts from the last to the first axis. If `axis` is a tuple of ints, a sum is performed on all of the axes specified in the tuple instead of a single axis or all the axes

as before.

`dtype` : dtype, optional
The type of the returned array and of the accumulator in which the elements are summed. The dtype of `'a'` is used by default unless `'a'` has an integer dtype of less precision than the default platform integer. In that case, if `'a'` is signed then the platform integer is used while if `'a'` is unsigned then an unsigned integer of the same precision as the platform integer is used.

`out` : ndarray, optional
Alternative output array in which to place the result. It must have the same shape as the expected output, but the type of the output values will be cast if necessary.

`keepdims` : bool, optional
If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then `'keepdims'` will not be passed through to the `'sum'` method of sub-classes of `'ndarray'`, however any non-default value will be. If the sub-class' method does not implement `'keepdims'` any exceptions will be raised.

`initial` : scalar, optional
Starting value for the sum. See `'~numpy.ufunc.reduce'` for details.

`where` : array_like of bool, optional
Elements to include in the sum. See `'~numpy.ufunc.reduce'` for details.

Returns

`sum_along_axis` : ndarray
An array with the same shape as `'a'`, with the specified axis removed. If `'a'` is a 0-d array, or if `'axis'` is None, a scalar is returned. If an output array is specified, a reference to `'out'` is returned.

See Also

`ndarray.sum` : Equivalent method.
`add`: `'~numpy.add.reduce'` equivalent function.
`cumsum` : Cumulative sum of array elements.
`trapezoid` : Integration of array values using composite trapezoidal

rule.

mean, average

Notes

Arithmetic is modular when using integer types, and no error is raised on overflow.

The sum of an empty array is the neutral element 0:

```
>>> np.sum([])
0.0
```

For floating point numbers the numerical precision of sum (and ``np.add.reduce``) is in general limited by directly adding each number

individually to the result causing rounding errors in every step. However, often numpy will use a numerically better approach (partial

pairwise summation) leading to improved precision in many use-cases.

This improved precision is always provided when no ``axis`` is given.

When ``axis`` is given, it will depend on which axis is summed.

Technically, to provide the best speed possible, the improved precision

is only used when the summation is along the fast axis in memory.

Note that the exact precision may vary depending on other parameters.

In contrast to NumPy, Python's ``math.fsum`` function uses a slower but

more precise approach to summation.

Especially when summing a large number of lower precision floating point

numbers, such as ``float32``, numerical errors can become significant.

In such cases it can be advisable to use ``dtype="float64"`` to use a higher

precision for the output.

Examples

```
>>> import numpy as np
>>> np.sum([0.5, 1.5])
2.0
>>> np.sum([0.5, 0.7, 0.2, 1.5], dtype=np.int32)
np.int32(1)
>>> np.sum([[0, 1], [0, 5]])
6
>>> np.sum([[0, 1], [0, 5]], axis=0)
array([0, 6])
```

```
>>> np.sum([[0, 1], [0, 5]], axis=1)
array([1, 5])
>>> np.sum([[0, 1], [np.nan, 5]], where=[False, True], axis=1)
array([1., 5.])
```

If the accumulator is too small, overflow occurs:

```
>>> np.ones(128, dtype=np.int8).sum(dtype=np.int8)
np.int8(-128)
```

You can also start the sum with a value other than zero:

```
>>> np.sum([10], initial=5)
15
```

Nested function / inner function

You can define a function inside a function. Of course, it will be local to the *outermost* function.

```
In [32]: def nested_function(val:list|None=None):
        """
        nested_function will show how you can nest a function
        inside another function.

        If `val` is not a list, return None.
        Otherwise, return a new list with each element doubled.

        Parameters
        -----
        val: list|None (list OR None)
            a list to process; default to None

        Returns
        -----
        None if val is None, double_it(val) otherwise.
        """
        if not isinstance(val, list):
            return None

        def double_it(to_double:list):
            """Double the elements of a provided list"""
            return [x * 2 for x in to_double]

        return double_it(val)

# Check what nested functions return:
nested_function([1,2,3,4,5])
```

```
Out[32]: [2, 4, 6, 8, 10]
```

A function can return multiple values

```
In [33]: def return_two_values(val:list):
        """
        return_two_values will return the original list(val)
        and the squared values of the original list.

        Parameters
        -----
        val: list
            a list to process

        Returns
        -----
        val, squared values of val
        """
        return val, [x**2 for x in val]

# You can do this:
orig, transformed = return_two_values([1,2,3])
transformed
```

```
Out[33]: [1, 4, 9]
```

Many functions in fact return multiple values.

- `enumerate(iterable)` : index and value returned
 # See that i and v are returned?
 # i = 0 & v = 3 -> i = 1 & v = 4 -> i = 2 & v = 5
 for i, v in enumerate([3, 4, 5]):
 ...
- `numpy.linalg.eig(A)` : (eigenvalues, eigenvectors)
 # A = $\begin{bmatrix} 3 & 4 \\ 7 & 11 \end{bmatrix}$
 A = np.array([[3, 4], [7, 11]])
 # Eigendecomposition of the matrix A
 # It will return `EigResult` which is a tuple of
 # `eigenvalues` and `eigenvectors`
 np.linalg.eig(A)
- `numpy.where(condition)` : tuple of arrays
 B = np.array([1, 3, 7, 9, 11])
 np.where(B > 5)

```
In [34]: A = np.array([[3, 4], [7, 11]])
res = np.linalg.eig(A)
res
```



```
Out[34]: EigResult(eigenvalues=array([ 0.36675042, 13.63324958]), eigenvectors
              =array([[ -0.83525687, -0.35209031],
                     [ 0.54985995, -0.93596603]]))
```

```
In [35]: # numpy.where (condition) will return indices where condition is True
# If an array is 1-D, it will return row indices.

# This will return (array([0, 1, 2, 3, 4]), )
# because there's no column indices
np.where([1, 2, 3, 4, 5])
```

```
Out[35]: (array([0, 1, 2, 3, 4]),)
```

```
In [36]: # If there a matrix:
# |0 0|
# |3 7|
# row 1, columns 0 & 1 are non-zeros (True)
# Therefore, this function will return (array([1, 1]), array([0, 1]))
np.where(np.array([[0, 0], [3, 7]]))
```

```
Out[36]: (array([1, 1]), array([0, 1]))
```

```
In [37]: # Another matrix (B):
# |1 3 7 9 11|
# |2 4 6 8 10|
# Numbers greater than 5 are:
# 7 (=B[0, 2])
# 9 (=B[0, 3])
# 11(=B[0, 4])
# 6 (=B[1, 2])
# 8 (=B[1, 3])
# 10(=B[1, 4])
# Therefore, the returned value will be:
# ---row indices---          ---column indices---
# (array([0, 0, 0, 1, 1, 1]), array([2, 3, 4, 2, 3, 4]))
B = np.array([[1, 3, 7, 9, 11], [2, 4, 6, 8, 10]])
np.where(B > 5)
```

```
Out[37]: (array([0, 0, 0, 1, 1, 1]), array([2, 3, 4, 2, 3, 4]))
```

```
In [38]: # 3D array?

# C[0, :] = |1 2|
#           |3 4|
# C[1, :] = |5 6|
#           |7 8|
# C[2, :] = |9 10|
#           |11 12|
C = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]],
              [[9, 10], [11, 12]]])
```

```
# Finding indices where C % 2 = 1
# Basically, finding indices of odd numbers.
# Values where the condition is True are:
# 1 (=C[0, 0, 0])
# 3 (=C[0, 1, 0])
# 5 (=C[1, 0, 0])
# 7 (=C[1, 1, 0])
# 9 (=C[2, 0, 0])
# 11(=C[2, 1, 0])
# Then the returned indices will be:
# (array([0, 0, 1, 1, 2, 2]),
#  array([0, 1, 0, 1, 0, 1]),
#  array([0, 0, 0, 0, 0, 0]))
np.where(C % 2)
```

```
Out[38]: (array([0, 0, 1, 1, 2, 2]),
          array([0, 1, 0, 1, 0, 1]),
          array([0, 0, 0, 0, 0, 0]))
```

Putting into context - how can these be applied to your work?

Let's suppose you have a csv file containing several variables. If you want to plot them iteratively, you can combine a function and a for-loop to automate the process.

In the following example, I show how I generated a figure illustrating the absence of significant sex differences in some kinematic measures calculated from wearable sensor data collected from infants' legs. I prepare six empirical cumulative distribution functions (ECDFs), separated by the biological sex of participants.

```
In [39]: # This is the file I work with.
# Note: I usually define a path variable for better readability.
file_path = "/Users/joh/Documents/HBCD/WorkGroup Manuscript" \
            "/CODE/ANALYSIS/kinematics_left_average.tsv"
"""
In fact, this is closer to what I write:
>>> from pathlib import Path
>>> dir_path = Path("/Users/joh/Documents/HBCD" \
                    "/WorkGroup Manuscript/CODE/ANALYSIS")
>>> file_path = dir_path / "kinematics_left_average.tsv"
>>> kin_df = pd.read_csv(file_path, sep='\t')
`Path` object helps manage file paths
across different operating systems.
Hope I can prepare a tutorial on this later...
if you're interested, let me know!
"""
```

```
# See how different concepts we discussed are combined here?
# - separator specification (sep='\t')
# - usecols: only read columns that start with (x.startswith) 'avg_'
kin_df = pd.read_csv(file_path,
                     sep='\t',
                     usecols=lambda x: x.startswith('avg_') or
                     x in ['sex', 'participant_id'])
```

```
In [40]: # showing you the first 5 rows.
# This is a 'wide' dataframe with many columns.
# Do you see that columns 3-8 start with 'avg_'?
kin_df.head()
```

```
Out[40]:
```

	participant_id	sex	avg_movement_rate	avg_pkacc	avg_avgacc	avg_m
0	sub-4044959361	Male	939.197558	2.469221	1.201126	
1	sub-9656135401	Female	837.510415	2.700774	1.294568	
2	sub-8573430162	Male	1027.589755	2.748496	1.295048	
3	sub-4495347014	Male	1121.695276	2.130639	1.122392	
4	sub-1329355075	Male	816.007845	2.557845	1.229988	

```
In [41]: # Note. lambda function is an anonymous function:
# a function without a name.
# A more explicit way of writing the same function is:
def is_avg_col(x):
    return x.startswith('avg_')
# Then, you can use `is_avg_col` in `usecols` parameter:
kin_df2 = pd.read_csv(file_path, sep='\t', usecols=is_avg_col)
```

```
In [42]: # Check if kin_df2 is the same as kin_df
kin_df2.equals(kin_df)
```

```
Out[42]: False
```

```
In [43]: # Prepare a figure - this you saw in Week 3 notebook.
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
# You may want to checkout the extra notebook of Week 4
# for seaborn basics.
import seaborn as sns
```

Here what I prepare a function that will be reused: `ecdf` . See how the components of a function (`docstring` , `default value of a parameter`)

are introduced as well.

```
In [44]: def ecdf(ax, x, hue, data, xlabel, title, palette='deep'):
        """
        ecdf will prepare an empirical cumulative distribution function
        (ECDF) plot on the provided axis.

        Parameters
        -----
        ax: matplotlib.axes.Axes
            axis to plot on
        x: str
            column name for x-axis variable
        hue: str
            column name for hue variable
        data: pd.DataFrame
            data source for the plot
        xlabel: str
            label for x-axis
        title: str
            title for the plot
        palette: str
            color palette for the plot; default to 'deep'
        """

        # For more details of `sns.ecdfplot`, check:
        # https://seaborn.pydata.org/generated/seaborn.ecdfplot.html
        sns.ecdfplot(data=data, x=x, hue=hue, ax=ax, palette=palette,
                      linewidth=1.8, hue_order=["Female", "Male"])
        ax.set_xlabel(xlabel, fontsize=11)
        ax.set_ylabel('ECDF', fontsize=11)
        ax.tick_params(axis='both', which='major', labelsize=8)
        ax.set_title(title, loc="left", weight="bold", fontsize=11)
```

Ok, then I need to prepare titles so that they can be provided to the function:

`ecdf`.

```
In [45]: # preparing a 10x6 inch figure with 2 rows and 3 columns of subplots
        # hspace and wspace are the height and width spaces between subplots
        fig = plt.figure(figsize=(10, 6))
        gs = GridSpec(2, 3, figure=fig, hspace=0.4, wspace=0.3)

        # We are making `axes` dictionary to map
        # each column name to each subplot axis.
        # This way, we can loop through `axes` dictionary later.
        keys = kin_df.columns[2:]
        axes = {
            keys[0]: fig.add_subplot(gs[0, 0]),
            keys[1]: fig.add_subplot(gs[0, 1]),
            keys[2]: fig.add_subplot(gs[0, 2]),
            keys[3]: fig.add_subplot(gs[1, 0]),
            keys[4]: fig.add_subplot(gs[1, 1]),
```

```

keys[5]: fig.add_subplot(gs[1, 2]),
}

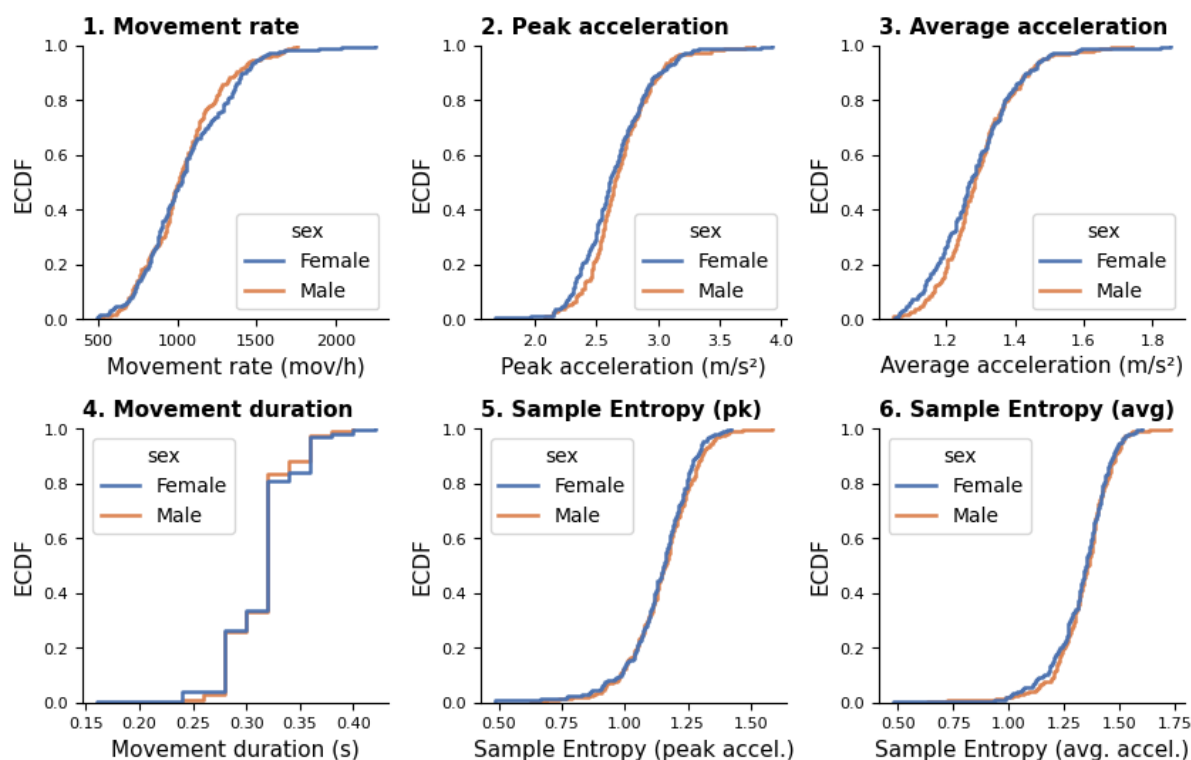
# 1. Prepare titles and xlabels separately
titles = [ "1. Movement rate", "2. Peak acceleration",
"3. Average acceleration", "4. Movement duration",
"5. Sample Entropy (pk)", "6. Sample Entropy (avg)"]

xlabels = ["Movement rate (mov/h)", "Peak acceleration (m/s²)",
"Average acceleration (m/s²)", "Movement duration (s)",
"Sample Entropy (peak accel.)", "Sample Entropy (avg. accel.)"]

# 2. Loop through axes and titles using `zip`
for key, title, xlabel in zip(keys, titles, xlabels):
    ecdf(ax=axes[key], x=key, hue="sex", data=kin_df,
        xlabel=xlabel, title=title)

# This command will remove the top and the right
# spines of all subplots.
sns.despine(fig=fig)

```



So you see that the you can use a function instead of writing multiple lines that are basically identical, differing only in their parameter values. Using for-loop to iterate over titles to labels is another key point to note.

Of course, this may limit your ability to customize each subplot, since the same style will be applied across all of them. For example, if you want to adjust the x-axis limit of subplot 5 only, you'll need to make additional tweaks. Still, this is an effective way to automate your workflow!

